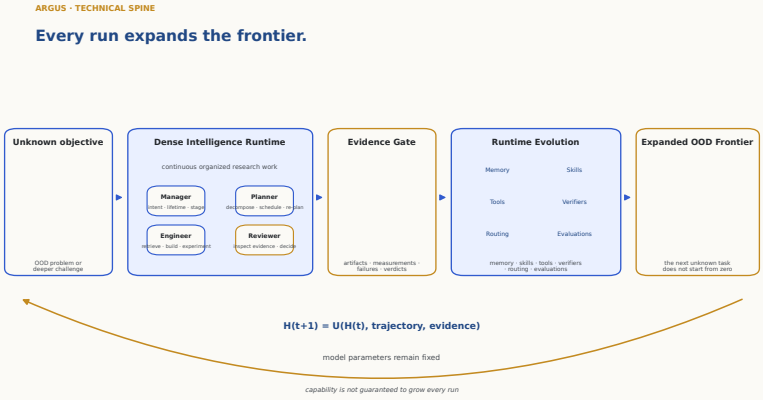
ARGUS

Autonomous Research Generation and Understanding System

Dense Intelligence for an Expanding Research Frontier



Argus is a research infrastructure that operates a capable coding-and-reasoning model as an autonomous research system over long horizons — hours to days — against real, public benchmarks and open research problems. This report documents the system as engineered: a three-plane architecture, the research runtime and mission lifecycle, the state and knowledge systems, the agent decision interfaces, the experiment and resource infrastructure, the reliability and recovery model, and the evidence architecture that keeps every measurement auditable. It reports the current public results across six arenas and a 41-paper research portfolio, each a scoped claim under its stated protocol and evidence tier. It is an engineering report, not a results paper: it does not claim universal state of the art, an accepted autonomous publication, or correctness outside the stated protocols.

Contents

ACT I · THE DENSE-INTELLIGENCE PROBLEM

Research intelligence is sparse in time.

Long-horizon research requires continuity, execution, verification, and durable state—not merely a longer context window.

1 Executive Thesis

Argus sustains dense research intelligence, turns each run's evidence into runtime capability, and carries that capability into the next unknown task. Every run expands the frontier.

Model intelligence is sparse in time. A capable coding-and-reasoning model is brilliant for the length of a single call and then stops: the reasoning that produced an insight evaporates when the context is dropped or lossily compacted, and the next call begins again with no durable trace of what was learned. Long-horizon research is the opposite of a single call. It is thousands of coupled decisions sustained over hours to days, where the value is not in any one clever step but in keeping judgment, execution, and verification connected across all of them. A longer context window does not close this gap; it only postpones the moment the episode ends.

Argus makes intelligence *dense* by running four persistent, model-driven roles as a continuous loop over persisted project state. A Manager fixes intent and lifetime, a Planner decomposes and schedules, an Engineer retrieves and builds and experiments, and a Reviewer inspects the evidence and decides. Because the loop never dissolves back into a single stateless call, decision, execution, and verification stay coupled: the system can carry a thread of research reasoning far past the horizon at which an episodic agent would have forgotten why it started.

Continuity alone is not enough — it must compound. Every run deposits verified evidence, and that evidence updates the system's *runtime state*: its memory, skills, tools, verifiers, routing, and evaluations. This update is gated by the Reviewer, so only checked results change the runtime, and it happens with the model's parameters held fixed — $H(t{+}1) = U(H(t), \text{trajectory}, \text{evidence})$. The system grows more capable not by retraining a model but by accumulating audited, reusable capability around it.

The consequence is out-of-distribution reach. When the next task arrives, the runtime it inherits already carries the skills, tools, and verifiers earned on prior problems, so the next unknown objective does not start from zero. Each expansion of verified capability enlarges the frontier of problems the system can attempt — the expanding-frontier chain of Figure 1.

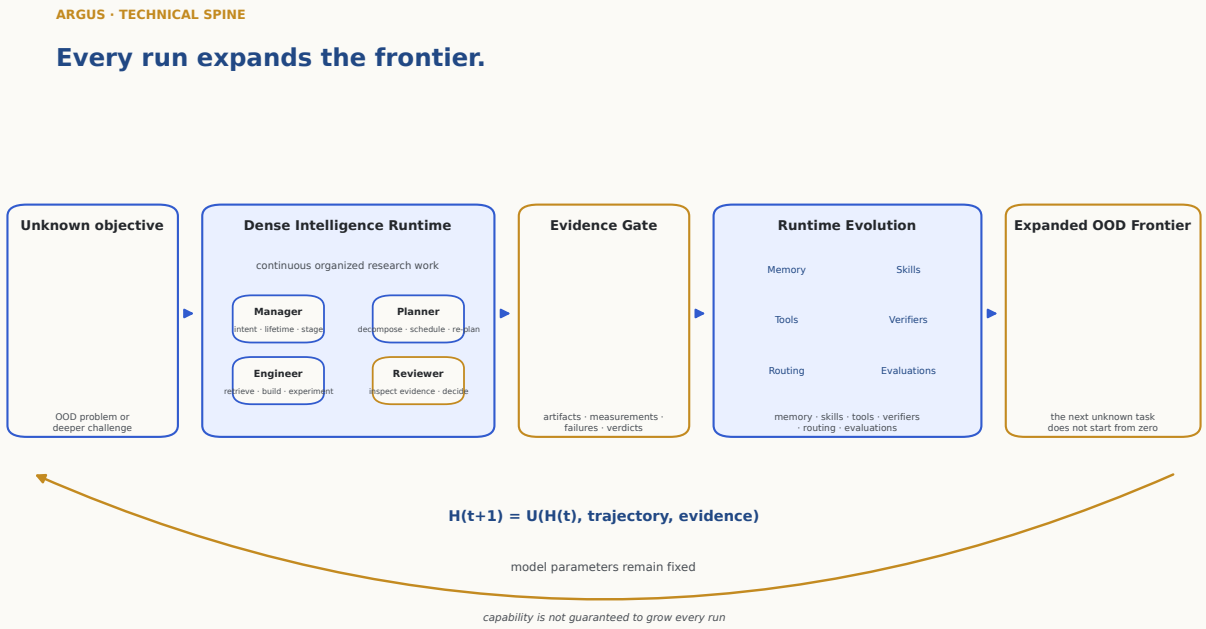


Figure 1. The expanding-frontier spine. An unknown, out-of-distribution objective enters a dense-intelligence runtime driven by four persistent roles; verified work passes an evidence gate; the gate updates durable runtime state; and the enlarged runtime meets the next unknown task from a higher floor. Model parameters remain fixed, and capability is not guaranteed to grow on every run.

What this report claims	What it does not claim
Argus is engineered as a persistent, role-separated runtime that keeps decision, execution, and verification coupled over long horizons.	That the infrastructure is more capable than the model, or substitutes heuristics for the agent's scientific judgment.
Each run's reviewer-verified evidence updates durable runtime state — memory, skills, tools, verifiers, routing, evaluations — with model parameters fixed.	Monotone empirical capability growth: a run may fail, return only a negative result, or add nothing verified.
Six public arena results, each a scoped claim under its stated protocol, hardware, and evidence tier.	Universal state of the art, or superiority over all human researchers or other systems.
A 41-paper portfolio compared only to human-authored literature, reported as a de-duplicated inventory.	An accepted autonomous publication, or a count of accepted papers.
Reliability and auditability by construction: bounded budgets, one completion authority, an append-only evidence tape.	Correctness, safety, or performance outside the exact protocols each result is measured under.

Table 1. The claim boundary the rest of the report is held to.

Argus publishes a live public record [1]. This report reproduces its six current arena results (Section 12, Figure 7), each with the protocol and hardware it was measured under and its published reference: NVIDIA SOL-ExecBench (B200, 101 kernels; global rank #6 with two head-to-head wins over Recursive [7]), nanochat on B200 (0.9636 BPB vs. human SOTA 0.9646) and H100 (0.9855 vs. 0.9879 BPB), the nanoGPT speedrun (79.77 s vs. the same-device human record 80.18 s, N=10), AARRI-Bench (63/82, 76.8% vs. the paper-reported best 68.3%), and Arbor's site-reported system comparison; two of the six — the nanoGPT speedrun and nanochat on B200 — additionally

carry committed artifact-digest records, while the other four remain website snapshots, and that distinction is kept visible throughout (Section 7). Separately, the public research page [2] lists a portfolio of **41 papers** — 35 manuscripts and 6 drafts across six research programs (Section 12.4, Figure 8) — compared only to human-authored literature and baselines, and reported as a de-duplicated inventory rather than a count of accepted papers.

Design objective, not a measured guarantee

"Every run leaves the system more capable" is the design objective: a run may fail, produce only a negative result, or add no verified capability. The report does not claim monotone empirical capability growth.

2 Dense Intelligence

Design objective

Keep research intelligence *dense* in time: hold decision, execution, and verification coupled and continuous over persisted state, so that a long-horizon program behaves as one sustained line of reasoning rather than a sequence of disconnected bursts.

2.1 Definition

By *dense intelligence* we mean sustained, coupled research work: at every point in a long run the system is making decisions, carrying them into real execution, and verifying the outcome against evidence — and the thread that connects these is never broken by the end of a model call. Sparse intelligence is the default failure mode of a capable model left to run long: it is brilliant in bursts and empty in between, because each burst ends and its working state is discarded. Density is not about being busy; it is about the fraction of elapsed research time in which genuine decisions are being made, executed, and checked, rather than lost to context recovery, idle polling, or unverifiable self-report. Figure 2 contrasts episodic research, where useful work is separated by context-recovery gaps, with the continuous role loop that runs over persisted project state.

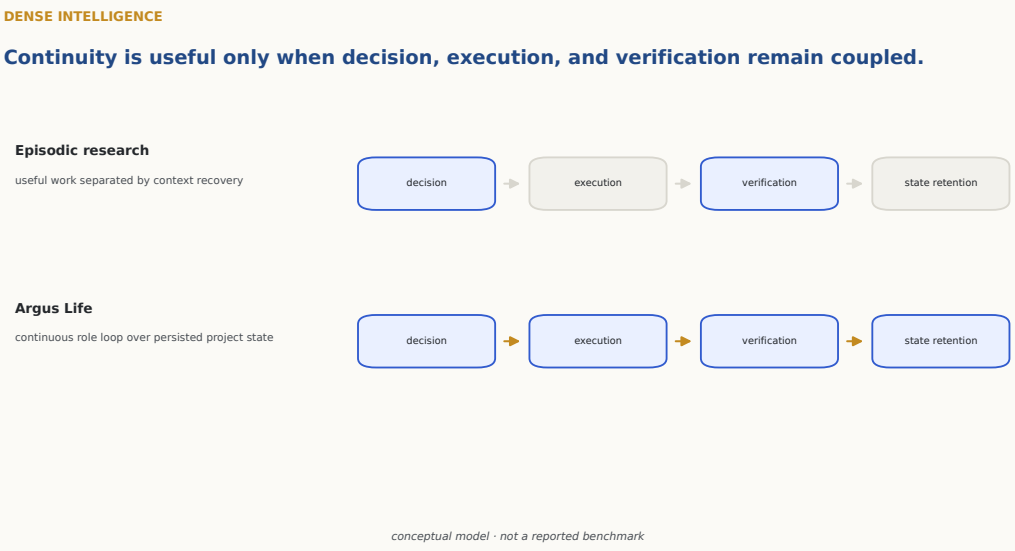


Figure 2. Dense intelligence as continuity of coupled work. Episodic research recovers decision and verification repeatedly across context gaps; Argus keeps decision, execution, verification, and state retention coupled over a persisted project. The schematic is a conceptual model, not a reported benchmark.

2.2 Continuity is not token volume

The tempting shortcut is to equate continuity with a larger context window: keep more tokens live and the run will stay coherent. It will not. A longer window lengthens a single episode but does nothing to couple decision to execution to verification, and it still terminates — at which point the accumulated working state is compacted or dropped and the next episode restarts cold. Token volume measures how much text a model can attend to at once; density measures how much of a research horizon is spent on verified, decision-bearing work. A run can consume enormous context and remain sparse — re-reading the same files, re-deriving the same dead ends — because none of that volume is gated by an independent check or retained as reusable capability. Continuity that matters is structural: it comes from persisting state outside the model and cycling roles over it, not from holding more tokens inside the model.

2.3 A conceptual density model

To make the idea precise enough to reason about — and no more — we summarize it as a time-averaged density over a research horizon T:

$$\rho_{\mathrm{DI}}(T) = \frac{1}{T}\int_0^T \lambda(t)\,\eta_d(t)\,\eta_x(t)\,\eta_v(t)\,dt. \tag{1}$$

The integrand multiplies a rate of research activity by three efficiencies, so that time spent busy but undirected, or active but unverified, contributes little.

Symbol	Meaning
T	The research horizon over which density is averaged — hours to days, not a single call.
$\lambda(t)$	Instantaneous rate of research activity: how much work the system is attempting at time t.
$\eta_d(t)$	Decision efficiency: the fraction of that activity that is a genuine decision advancing the frontier rather than repetition.
$\eta_x(t)$	Execution efficiency: the fraction of decisions carried into real execution on real tools, data, and hardware.
$\eta_v(t)$	Verification efficiency: the fraction of executed work that is independently checked against evidence.

Table 2. The five symbols of the conceptual density model.

Read this way, the episodic failure mode is exactly a collapse of one or more efficiencies to near zero for long stretches: high λ but low η_d (busy, not deciding), or non-trivial η_d and η_x but low η_v (acting, not verifying). The design of Argus is an attempt to hold all three efficiencies away from zero across the whole horizon by construction — persistent roles for decision and verification, real execution for η_x, and an evidence gate for η_v.

2.4 What is and is not measured

The model above is a lens, not an instrument. ρ_{DI} is an explanatory construct, not a reported benchmark metric. It does not, in itself, establish that Argus is universally superior to human researchers or other systems. No number in this report is a measurement of ρ_{DI}: the integrand's efficiencies are not instrumented as a live scalar, and we make no attempt to publish one. What *is* measured is reported elsewhere under explicit protocols — the six public arena results and the 41-paper portfolio of Section 12 and Section 12.4, each a scoped claim under its stated evidence tier. Dense intelligence is the design goal those results are evidence *for*; it is not itself a score, and it is not a claim of general superiority.

3 Why Episodic Agents Stall

Design objective

Diagnose why a capable model, run long, degrades into busy sparsity — and derive the design requirements that a system must meet to make research *compound* across runs instead of restarting from zero.

3.1 Model capability is not system capability

It is easy to assume that a more capable model is, by itself, a more capable research system. A stronger model does raise the ceiling of any single step: it reads code more accurately, writes better prose, and reasons further in one call. But a research *system* is judged over thousands of steps sustained across hours to days, and that horizon is governed by properties the weights do not supply: whether the thread of work survives the end of a call, whether claims are checked before they are believed, whether what was learned is retained and reused. Model capability is per-call; system capability is cross-call. Treating them as the same thing is why "give a big model a hard problem and wait" fails — the missing capability lives in the substrate around the model, not in the model.

3.2 Episodic work pays context-recovery cost

Left to run long, a model works in episodes. Each episode accumulates context until the window overflows and the transcript is compacted or dropped; the next episode then reconstructs where it was — re-reading the same files, re-deriving decisions it already made, re-discovering dead ends it already ruled out. This context-recovery cost is pure overhead: activity with near-zero decision efficiency. It is not a prompt-quality problem that a better instruction removes; it is a structural property of unbounded session growth. The longer the horizon, the larger the fraction of elapsed time the system spends paying this tax instead of advancing the frontier.

3.3 Unbounded context is not institutional memory

The obvious response — never drop context, just make the window bigger — solves the wrong problem. An ever-growing transcript is not memory. Institutional memory is curated, addressable, and reusable: a specific technique that worked, a baseline that was rejected and why, a verifier that caught a fake result. A monolithic transcript is none of these; it is an undifferentiated log in which the few reusable facts are buried among the reasoning that produced them, and it is precisely what lossy compaction shreds first when the window finally overflows. Durable capability has to live *outside* the model, in persisted state that can be written deliberately, retrieved selectively, and carried into a future run.

3.4 Process data disappears when only the final artifact survives

Most of the value of a research run is in its process, not its final artifact. The manuscript or merged patch records the destination; it discards the map of how the work got there — what was attempted and failed, which measurement was found contaminated, why one design was chosen over another. When a system keeps only the final artifact, that process knowledge is thrown away at the end of every run, so the next run cannot stand on it and is free to repeat the same mistakes. A system that is meant to compound must persist the trajectory and the verdicts, not just the deliverable, and must treat those process records as first-class, auditable state.

3.5 Design requirements for compounding research

These failure modes are not independent bugs; together they define what a long-horizon research system must provide. Five requirements follow directly, and each is met by a concrete mechanism in the architecture that Act II documents (Section 5).

R1. Persistent objective. A durable, operator-anchored objective that survives restarts and upgrades, so the system always knows what it is pursuing and never silently re-plans a campaign

from a stale premise.

R2. Role separation. Distinct model-driven roles for decision, execution, and verification, so no single stateless call owns all three and completion is never a self-report from the role that did the work.

R3. Bounded context. Per-session context held to an explicit bound and re-seeded from curated working memory, so the runaway compaction that drives context-recovery cost cannot occur.

R4. Independent evidence. Progress and completion judged by an authority that inspects artifacts and execution logs against evidence, rather than inferred from activity, filenames, or token volume.

R5. Reusable runtime state. Verified evidence deposited as durable, addressable capability — memory, skills, tools, verifiers, routing, evaluations — that the next unknown task inherits instead of starting from zero.

Act II turns these five requirements into a running system: a three-plane architecture that separates decision authority, execution, and evidence, and the runtime that keeps them coupled over a persisted project.

ACT II · THE MACHINE THAT TURNS WORK INTO CAPABILITY

Continuity requires an institution, not a longer prompt.

Argus organizes persistent objectives, bounded role contexts, real execution, independent review, and durable evidence into one research runtime.

4 Argus Life and the Technical Spine

Design objective

Turn the report's master spine — an unknown objective entering a dense-intelligence runtime, passing an evidence gate, updating durable runtime state, and meeting the next task from a higher floor — into named, source-grounded components. Argus Life keeps the lifetime objective moving; each mission is a bounded unit with a first-class outcome.

Act I (Section 3) derived five requirements for a compounding research system: a persistent objective, role separation, bounded context, independent evidence, and reusable runtime state. Act II is the running system that meets them. This section is the map from the expanding-frontier spine (Figure 1) to the implementation — which component owns each stage and what durable artifact it leaves behind — and the sections that follow expand each stage: the four roles and three planes (Section 5), the mission lifecycle and durable state (Section 6), the evidence and review architecture (Section 7), and the reliability and resource machinery (Section 8).

4.1 Life keeps the objective moving; missions stay bounded

Argus Life is the persistent context of a campaign: a durable, operator-anchored objective and its committed configuration, held on disk and recovered across restarts and upgrades. A `STANDING` objective is persisted atomically to `continuous.json` and is never silently re-decided (Section 6). Life itself issues no provider work; it exists so a long campaign has a stable identity. The unit of *work* is a mission: one bounded Engineer↔Reviewer run that always ends in a first-class terminal or paused status, never an unbounded spin. Life supplies continuity; missions supply bounded, auditable progress. That division is what lets the system run for days while every increment stays a discrete, inspectable event.

4.2 The spine, mapped to owners and durable outputs

Table 3 maps each stage of the master spine to the component that implements it and the durable artifact it writes. The mapping is deliberately one-to-one: every conceptual stage has exactly one implemented owner and one place its result is persisted, so a reader can move from the figure to the code without guessing.

Two properties of the mapping matter. First, the *evidence gate* is a single role — the Reviewer — not a hardcoded check (Section 5); the authoritative record of a verified achievement is exactly one event on the tape, `research.achievement.certified` (Section 7). Second, *runtime evolution* is not one mechanism but several, and the report is careful not to flatten them into a single arrow.

4.3 Runtime evolution has four distinct owners

The spine's runtime-evolution step — the update $H(t{+}1) = U(H(t), \text{trajectory}, \text{evidence})$ with the model's parameters held fixed — is realized by four ownership paths, each carrying a different authority.

Master-spine concept	Implemented owner	Durable output
Unknown / OOD objective enters	Manager front door + division (`manager/_core.py`)	committed vertical + lifetime in `continuous.json`
Dense-intelligence runtime	`LifeSupervisor` outer loop (`life/supervisor/_core.py`) + `SkillLoop` per mission (`loop.py`)	`events.jsonl` tape + project journal
Four persistent roles	Manager, Planner, Engineer, Reviewer	typed role verdicts on the tape
Evidence gate	Reviewer completion authority (`reviewer/_core.py`)	`research.achievement.certified`
Runtime evolution	Reviewer `skill_ops`/`wiki_ops`; Scientist distill-on-miss (`skills/`, `wiki/`)	versioned skills + immutable wiki sources
Expanded frontier / next task	`LifeSupervisor` next claim; Planner refill (`planner/planner.py`)	refilled `backlog.jsonl` inherited by the next mission

Table 3. The master spine (Figure 1) mapped to implemented owners and their durable outputs. Each stage has one owner and one persistence point.

Automatic.
Some updates are mechanical. On a matcher miss a Scientist distills an immediately usable project-layer skill; after every real Reviewer verdict a *source* wiki RoundCard is written verbatim from the verdict; and a recency memory preamble is rendered before each mission. None requires further judgment (Section 6).

Model-proposed.
The Engineer *proposes* — it drafts new skill content and, at a turn boundary, emits a curated working-memory handoff. A proposal is a candidate, not yet a committed change.

Reviewer-authored.
Durable knowledge is *committed* by the Reviewer, the sole author of `skill_ops` and `wiki_ops` and the validator of the canonical `checkpoint.json`. A synthesized wiki page is admitted only when its evidence spans verify verbatim against a source (Section 7).

Operator-invoked.
Some evolution is human. An operator edits the machine identity, exports or overwrites skills, sets configuration knobs, switches a role's backend through the Manager, or leases GPUs — explicit actions, never inferred by the runtime (Section 8).

Keeping these four distinct is a design commitment: the system never presents an automatic distillation as if a human authorized it, nor a Reviewer-certified skill as if it were an unchecked model suggestion. Authority over the runtime is always attributable.

5 Four Roles, Three Planes, One Research Runtime

Design objective

Separate authority two ways at once. Three planes physically divide decision, execution, and evidence, so judgment and record-keeping can never be confused. Four persistent roles carry every scientific and scheduling decision across a narrow typed interface — a Python dataclass with named fields — so no role can quietly reach past its contract.

Argus is one research runtime organized as three cooperating planes and driven by four persistent roles. The **control plane** holds the roles and scheduler that decide *what to run and whether it is done*; the **execution plane** runs *one mission at a time* against real tools, providers, and budgets;

and the **evidence plane** records the run on an append-only tape and never decides it. Figure 3 shows the planes and their interfaces, and Figure 4 is the accepted schematic with the four roles inside the persistent runtime. Table 4 assigns each plane its authority.

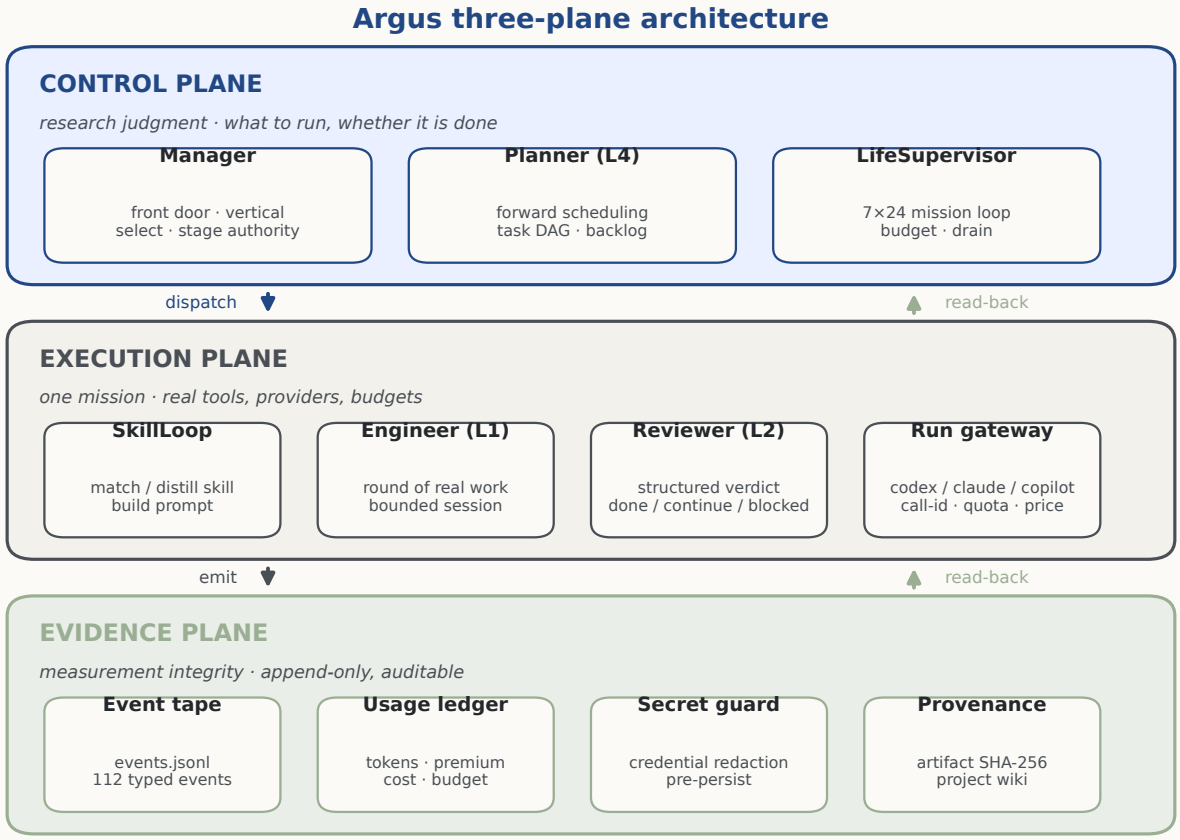


Figure 3. The three-plane architecture. Control-plane roles dispatch work into the execution plane, which emits typed events into the evidence plane; the evidence plane is read back by both other planes but holds no decision authority. Drawn deterministically from the module map (Appendix E).

The end-to-end control path is a single spine: `CLI` → `Manager.divide` → runtime wiring → `LifeSupervisor` → `SkillLoop` → `SupervisedEngineer` ↔ Reviewer-until-done, resting on the core services, the single anti-stall mechanism, the pluggable verticals, and the provider backend. Every provider call, regardless of role, is issued through one application-level gateway (`core/run_gateway.py`) that assigns a `call_id` and stamps timing — the single interception point that makes tracing, budgeting, and the per-call audit of Section 7 possible. The concrete backends (Codex, Claude Code, Copilot CLI) sit behind an adapter; a deterministic in-memory backend drives tests.

The four roles below each emit a typed decision interface, so authority is explicit and verdicts are machine-checkable. Three form a numbered execution spine (Planner L4, Engineer L1, Reviewer L2); the Manager spans the whole pipeline and sits outside the numbering. (A fifth role, the Curator, runs only in team/subagent mode and is out of scope here.)

5.1 Manager — front door and stage authority

The Manager converts operator free text into a committed campaign in a single grounded model call that emits three structured axes at once — `CONFIG` (e.g. a backend/model change), `CONTROL` (whether to dispatch at all), and `ROUTE` (chat vs. task and which vertical). A `CONTROL: NO_DISPATCH` decision is authoritative: the bridge forces a read-only self-reply and, if that fails, fails closed rather than enqueuing work. A task division yields a `Division`; stage progression is a `StageTransition` (Table 5). The Manager is the sole writer of `current_stage`; the transition's `action` and `source` fields make both the decision and its justification auditable, including the fail-safe holds that keep

Plane	Primary components	Authority
control	Manager, Planner, `LifeSupervisor`, daemon (`manager/`, `planner/`, `life/`, `daemon/`)	decides what to run and whether a project is done
execution	`SkillLoop`, Engineer, Reviewer, provider gateway, verticals (`loop.py`, `engineer/`, `reviewer/`, `core/run_gateway.py`)	runs one mission; the Reviewer holds mission-level scientific authority
evidence	event tape, usage ledger, secret guard, provenance, wiki (`core/`, `skills/provenance.py`, `wiki/`)	records and checks; holds no decision authority

Table 4. The three planes and their authority. Control and execution decide; evidence records. Full module map: `docs/ARCHITECTURE.md`.

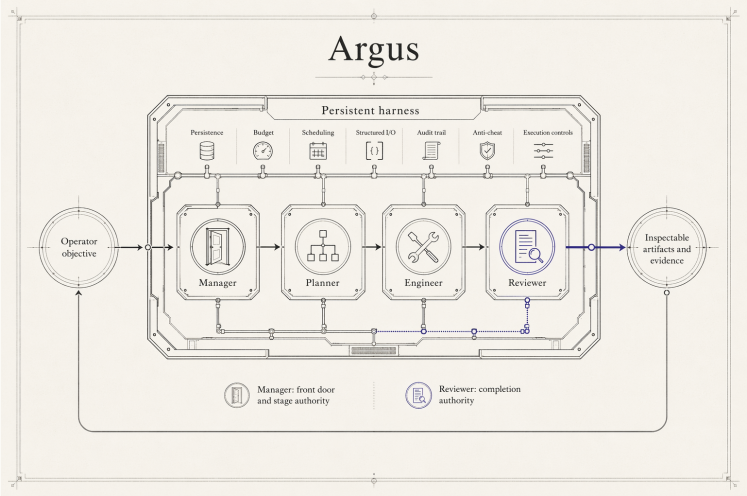


Figure 4. System schematic (not a measured performance plot): an operator objective enters the persistent runtime; the Manager, Planner, Engineer, and Reviewer sit inside it as the four roles that carry every research judgment; and inspectable artifacts and evidence exit back to the operator.

the pipeline coherent when a downstream role is unavailable. The other roles may advise a stage change but never write it.

5.2 Planner (L4) — forward scheduling

When the backlog empties in continuous mode, the Planner proposes the next batch of work as a set of `TaskSpecs` and returns a `PlannerVerdict` (Table 6). Tasks form an optional DAG — each `TaskSpec` has a local `key` and a `deps` list the scheduler maps to real backlog ids — and each carries a structured `scope` threaded to the Reviewer as a backlog tag, so completion authority never re-parses prose to learn a task's scope. The verdict also has a first-class `waiting` outcome (with a durable `WaitingContract`) for a project correctly blocked on a live external job with no genuinely new high-impact work to queue. The Planner is the sole plan author; the Reviewer may only report that new evidence has overtaken the current plan.

5.3 Engineer (L1) — one round of real work

The Engineer executes a single round against real tools and hardware. Its interface to a provider is the per-call `RunnerOptions` contract, and it receives back a `RunnerResult` (Table 7). Two properties matter for integrity. First, provider-side spend fences (`max_budget_usd`, `max_ai_credits`)

Field	Type / values	Meaning
Division `manager/_core.py`		
`task`	str	the committed objective text
`vertical`	str	the one selected task shape (never re-classified)
`stages`	list	the vertical's ordered stage template
StageTransition `manager/_core.py`		
`action`	advance | hold | rollback | complete	the move applied to `current_stage`
`source`	manager_llm | *_hold	a model decision, or a fail-safe hold

Table 5. Manager decision interface. Only the Manager mutates `current_stage`; a `hold` writes nothing.

Field	Type	Meaning
TaskSpec `planner/planner.py`		
`title, objective`	str	human title + full actionable description
`impact_score`	int 0–5	parser accepts only high-value work
`scope`	bounded | final_submission	threaded to the Reviewer as a tag
`key, deps`	str, list	DAG node id and its dependencies
PlannerVerdict `planner/planner.py`		
`project_done`	bool	project-level completion proposal
`new_tasks`	list	the next batch
`waiting`	bool	intentional idle on a live external blocker
`restart_daemon`	bool	request a controlled daemon restart
`checklist_ops`	list	per-stage checklist edits (Planner owns it)

Table 6. Planner decision interface (abridged). Scope is structured, never inferred from the objective text.

are populated from an atomic budget *reservation*, not from role configuration, so a role cannot raise its own ceiling (Section 8). Second, the result carries usage-*presence* booleans, so a real zero token count is distinguishable from a provider that simply omitted usage — the evidence plane never invents a number a provider did not report. The Engineer's round-loop reliability guards are documented in Section 8.

5.4 Reviewer (L2) — the completion authority

The Reviewer is the sole authority on completion, and there is no separate hardcoded completion gate anywhere in the system. Each round it inspects the Engineer's output against a structured stage checklist and returns a `ReviewDecision` (Table 8) whose `status` is `done`, `continue`, or `blocked`. Because the Reviewer runs in the same work-tree as the Engineer and is given the mission's execution log with grep recipes, it can audit *how* a result was produced — catching a hardcoded answer, a skipped step, or a faked metric — rather than trusting a summary (Section 7). Its `progress_class` is a structured judgment of forward motion that the infrastructure *counts* but never infers; `skill_ops` and `wiki_ops` are authored here with no Manager gate; and a `final_submission_certified` property is fail-closed, true only when the structured gate is genuinely satisfied.

Why "done" has exactly one owner

Premature acceptance (Section 3) is answered structurally: completion is a single typed verdict from a role that sees the artifacts and the execution log, not a self-report from the role that did the work, and not a keyword the infrastructure fished out of a summary. A prior reviewer path

Field	Meaning
RunnerOptions (per-call knobs) `core/models.py`	
`model, reasoning_effort`	provider model and effort level
`output_schema_path`	structured-output schema for the call
`sandbox_mode`	containment policy for the spawned role
`max_budget_usd, max_ai_credits`	spend fences from a reservation, not config
RunnerResult (per-call outcome) `core/models.py`	
`call_id, call_id_log_correlated`	audit correlation to the event tape
`*_tokens_present`	presence flags: a real zero vs. an omitted usage
`cost_usd, pricing_status`	usage cost and pricing state

Table 7. Engineer per-call backend contract (abridged). Spend fences come from a reservation; presence flags keep usage accounting honest.

Field	Meaning
`status`	`done` | `continue` | `blocked` (the completion verdict)
`progress_class`	`decision` | `evidence` | `setup_only` | `artifact_sync_only` | `none`
`next_action`	concrete next step handed to the subsequent Engineer round
`operator_question`	the one human-facing block question, if any
`scope, checklist`	final-submission gate (fail-closed)
`skill_ops, wiki_ops`	knowledge-system edits (Reviewer is sole author)
`backend_unavailable`	infrastructure-failure flag, not a judgment

Table 8. Reviewer verdict — the completion contract (`core/models.py`, `reviewer/reviewer_schema.json`).

that scanned the Engineer's summary with regexes and forcibly rewrote a `done` into `continue` was removed; the constraint now lives in the Reviewer's own prompt. If the Reviewer is wrong, that is a model-quality limitation (Section 13) — but the authority is never ambiguous, and it is never silently overridden.

6 Mission Lifecycle and Durable State

Design objective

Run one mission at a time, back to back, for days — surviving restarts, upgrades, and provider outages — without ever re-deciding a committed campaign or killing a mission mid-flight. Keep the durable truth on disk: the append-only event tape is the system of record, and every other file is a derivable view. Provider sessions are bounded caches, not the sole memory.

The runtime is the 7×24 loop that turns a durable objective into a stream of missions. Figure 5 shows the lifecycle and its recovery edges. This section describes the states, the process model, the restart/resume semantics, and the on-disk state that together make a long campaign durable.

6.1 From backlog claim to mission outcome

The `LifeSupervisor` runs a synchronous outer loop: it claims the next backlog item, injects a non-authoritative recency memory preamble without polluting the raw objective, runs the mission

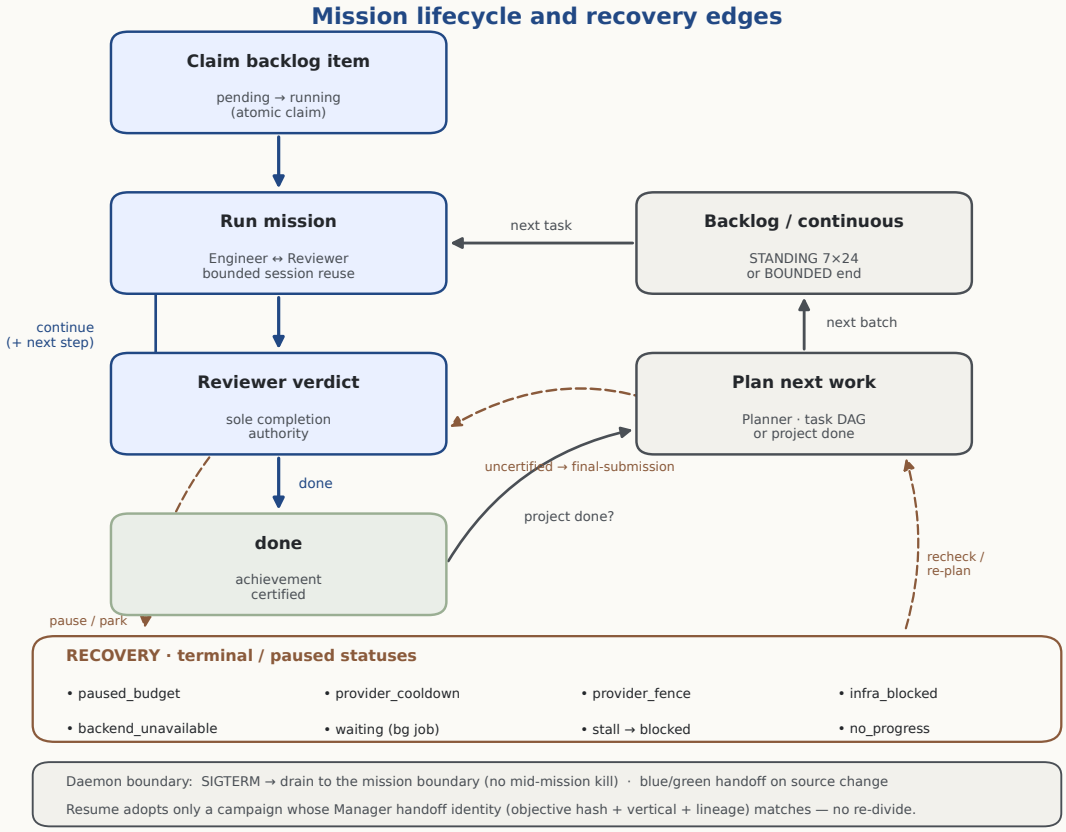


Figure 5. Mission lifecycle and recovery edges. The primary spine (indigo) claims a backlog item, runs an Engineer↔Reviewer mission, and acts on the Reviewer verdict; the plan-next loop (grey) refills the backlog; recovery edges (terracotta) route paused and blocked missions to first-class statuses and back. The daemon boundary governs draining, handoff, and identity-checked resume.

through the execution plane, accounts the cost against budget gates, writes a journal entry, and — when the backlog empties in continuous mode — invokes the Planner for the next batch. The backlog claim (`pending`→`running`) is atomic, so two workers can never take the same item; orphaned `running` items from a prior crash are reaped on startup. The loop is deliberately synchronous — one mission at a time — which is what makes the mission boundary a clean, well-defined point for stopping, upgrading, and resuming.

Every mission ends in a first-class terminal or paused status, never an unbounded spin. Terminal outcomes include `done`, `blocked`, `max_rounds`, `no_progress`, and `error`; paused outcomes include `paused_budget`, `paused_provider_cooldown`, `paused_provider_fence`, and `infra_blocked` (full taxonomy in Appendix B). A paused mission is not a failure — it records that the campaign is correctly waiting on an external condition. Separately, a `replan_requested` status is a *control transfer* back to the Planner, not a successful completion and not a done/failed/blocked outcome: the current item returns to `pending` and then passes the normal Planner rate and budget gate before any re-plan takes effect.

6.2 Persisted identity and safe resume

For unattended operation the runtime runs as a detached daemon worker (`daemon/life_worker.py`, roughly 1,800 lines) wrapping the supervisor. On `SIGTERM` or `SIGINT` it drains to the next mission-tick boundary rather than interrupting a live mission, so an in-progress evaluation or a result being recorded is never severed. A run is protected by both the atomic backlog claim and a per-project `daemon.pid` lock, so a second daemon cannot drive the same project.

A 7×24 system must be upgradable without losing a campaign. Argus handles a source change with a **blue/green self-handoff**: the worker detects a change in its own source signature, spawns a candidate child under a handoff protocol with a bounded generation count and a minimum inter-

Path (per project)	Owner	Content	Mutability
`events.jsonl`	evidence plane	canonical typed-event timeline	append-only
`backlog.jsonl`	scheduler	pending / running / done work items	atomic claim
`continuous.json`	scheduler	STANDING lifetime + campaign config	rewrite
`inbox.jsonl`	operator	queued operator messages / answers	append + drain
`checkpoint.json`	Reviewer	hard-capped curated working memory	atomic rewrite
`daemon.pid`	daemon	concurrency lock	exclusive
`daemon.status.json`	daemon	liveness / status snapshot	rewrite
`identity.md`†	global	author / machine identity	operator
`skills/`†	global	shared skill library	versioned
`config.json`†	global	operator knob overrides	operator

Table 9. Principal persisted state (abridged; full map in Appendix C). The event tape is the only append-only source of truth; everything else is derivable or resettable. † under the global root `~/.argus-skill/`.

handoff interval, and hands the campaign over at a clean boundary (`daemon/handoff.py`). The Manager-to-daemon handoff *identity* (objective hash, vertical, lineage generation) is persisted and recoverable from the immutable event tape. Resume is identity-checked: a `-resume-continuous` start adopts only a persisted campaign whose identity matches; a mismatched or missing identity forces a real Manager division rather than blindly re-attaching. Crash and reboot restart is owned by the `systemd -user` unit (Section 8) plus `loginctl enable-linger`, not bespoke keep-alive logic.

6.3 Bounded sessions and curated working memory

Argus does *not* depend on one unbounded provider transcript. The Engineer and Reviewer use separate resumable sessions, so recent context and provider prefix caches stay useful, while a structured `checkpoint.json` in the project state directory carries curated cross-round state (`engineer/checkpoint.py`, `apps/_runtime.py`). Its `CheckpointState` fields cover the goal, verified work, dead ends, one open blocker, the next step, and bounded environment facts. The *Reviewer* authors it: the Engineer ends a turn with a concise handoff proposal, and the Reviewer validates that proposal against the artifacts and writes the canonical checkpoint by atomic rewrite. Python enforces hard item and character caps — not only in the prompt or schema — so the checkpoint stays a *current-state* object rather than an append-only log; stale detail is deleted while ground truth remains in on-disk artifacts.

The resume window is explicitly bounded. By default the runtime rolls each role's thread after three rounds on that thread, or when the preceding call reports at least 1.5 million input tokens; the next call opens a new session and the Engineer is reseeded from the checkpoint (`engineer/runner.py`, `core/knobs.py`). If provider-side compaction occurs before a roll, the next round resends the full static preamble. This combines short-window context reuse with a structural limit on transcript growth, and it fails soft: a missing or garbled checkpoint keeps the last good one rather than clearing memory.

6.4 The on-disk state map

Core path helpers define the global and per-project roots (`core/paths.py`); runtime composition derives the checkpoint path inside the project state directory (`apps/_runtime.py`). A global root `~/.argus-skill/` (overridable via `ARGUS_SKILL_HOME`) holds cross-project identity, shared skills, and an operator knob store; each project lives under `projects/<fingerprint>/`. Table 9 lists the principal state files, their owner, and their mutability. The canonical timeline is the append-only `events.jsonl`; every other file is a derived view or working state that can be rebuilt or safely reset.

Long-term knowledge lives in two systems on this state. Skills are versioned Markdown files distilled

on a matcher miss and revised from real usage (`skills/`); a per-project wiki records an immutable *source* RoundCard after each Reviewer verdict, from which the Reviewer may later synthesize evidence-cited pages (Section 7). Both compound capability across missions without becoming an unbounded prompt.

7 Evidence and Independent Review

Design objective

Make every reported number reconstructable and every completion auditable. The evidence plane records the run on an append-only tape, lets the Reviewer audit how each result was produced, redacts credentials before persistence, hashes published artifacts for provenance, and labels every quantity by the strength of its evidence. A verdict may be wrong, but it can never be unfalsifiable.

7.1 The immutable event tape

The canonical record of a project is an append-only event tape, `events.jsonl`, written through typed events from a catalog of 112 event types across 11 categories, of which 75 carry a validated payload schema (`core/event_catalog.py`; Appendix D). Within it exactly one event, `research.achievement.certified`, is the authoritative record of a verified achievement, emitted only on a `done` verdict carrying the structured achievement payload (Section 5). A campaign's decisions, spend, and outcomes can be replayed from the tape after the fact, which is what makes an audit possible at all.

7.2 Execution-log audit and call correlation

Trusting a self-report would defeat the completion contract, so the Reviewer checks the Engineer's work directly. The two roles share a work-tree, and the Reviewer's prompt includes an audit section pointing at the mission's own `events.jsonl` with grep recipes (`engineer/runner.py`), so the Reviewer can inspect *how* a result was produced — detecting a hardcoded answer, a skipped step, a cheat method, or a faked metric — rather than accepting the summary. Each provider call is correlated to the tape by `call_id`, and usage-presence booleans keep a real zero distinct from a missing measurement (Section 5). This verifies the number is honest, not that the science is good.

7.3 Anti-cheat by construction

For benchmarks whose metric could be gamed by memorizing a fixed evaluation input distribution, the evaluation input is randomized per run, so a solution cannot hardcode the statistics of a known distribution — a fast-because-faked kernel is designed to be impossible to pass off as genuinely fast. Combined with real public benchmark data and the per-round audit trail, this closes the metric-exploitation failure mode by construction rather than by after-the-fact inspection. Integrity is per-benchmark and remains an ongoing surface: a new benchmark can present a new exploit the current checks were not written for (Section 13).

7.4 Credential redaction before persistence

A task-agnostic secret guard (`core/secret_guard.py`) scrubs credentials from newly written artifacts before they reach the event log or the Reviewer's prompt, and redacts persisted event and usage copies, matching authorization and API-key patterns, provider tokens, and generic key/value secrets. Redaction never judges scientific quality; when it rewrites an artifact it emits `round.secret_redacted` so the Reviewer can rebuild the affected provenance hashes, and a scan error or an oversized artifact explicitly *blocks* unconditional certification rather than silently passing — the same fail-closed stance as the completion gate.

7.5 Provenance, snapshots, and digests

Published evidence carries provenance. For website-sourced results, the retrieval records an HTTP status and a raw-HTML SHA-256 for each source page; for corroborated results, the bundle identifies an artifact in a named external project by logical project name, artifact id, and SHA-256 — with no local absolute paths, usernames, or session ids. Table 10 separates a `website_snapshot` — a result quoted verbatim from the live public record — from a result that additionally carries committed external-project artifact-digest metadata. The artifact bytes are not committed in this repository (`technical_report/evidence/README.md`). An optional, off by default Ed25519 result-signing path (`team/result_provenance.py`) lets an isolated scorer sign a teammate's result so a forged or tampered value fails verification. It is wired for isolated-scorer teammate results, but it is not used for the results reported here, and this report's evidence chain does not rely on it.

Corroboration / category	What it means	What it may claim
`local_artifact`	A public result with a committed artifact-digest record for a named external project (logical project + artifact id + SHA-256).	An Argus result for that run, on stated hardware.
`website_snapshot`	A result card quoted verbatim from the live public record, with no artifact-digest corroboration yet.	A scoped public claim under its stated protocol.
External baseline	A published third-party or human target used for comparison.	A reference target; explicitly *not* an Argus result.
Ongoing / no claim	A program running without certified, reproducible completion.	Status "ongoing"; no numeric Argus claim.

Table 10. Evidence categories. All six public results originate as `website_snapshot`; the bundle's separate `corroboration` field marks two as `local_artifact` because artifact-digest metadata is available (Section 12).

7.6 Completion is a single certified verdict

The measurement-integrity architecture converges on one rule: a program is complete only when the Reviewer certifies it against the full-pipeline stage checklist, with the `final_submission_certified` property fail-closed. There is no hardcoded completion gate and no independent validator behind the Reviewer; an earlier family of roughly twenty machine "validate-*" quality gates was removed once completion authority moved to the Reviewer's checklist, so completion has exactly one owner and one auditable record. This concentrates authority in a single decision point — examined honestly in Section 13, since the Reviewer is a model with no independent oracle behind it — but it also means completion is never the product of two components silently disagreeing.

8 Reliability, Resources, and Operation

Design objective

Treat execution drift — not weak reasoning — as the primary risk of a multi-hour run. Every degradation mode maps to a named guard that fails soft and is precise about whether it may interrupt a live call. Route every provider call through one priced, reservation-backed gateway so spend is bounded and attributable. Give the operator a live, authenticated cockpit over the same tape, and deploy as a standard service that drains cleanly.

The reliability model is a set of guards that keep a mission bounded and honest. Figure 6 sketches how they wrap one Engineer/Reviewer loop over a persistent budget, event-log, and artifact rail. None of these guards decides whether the research is good: each enforces a task-agnostic invariant

and fails soft — reliability is added *around* the agent's judgment, never in place of it.

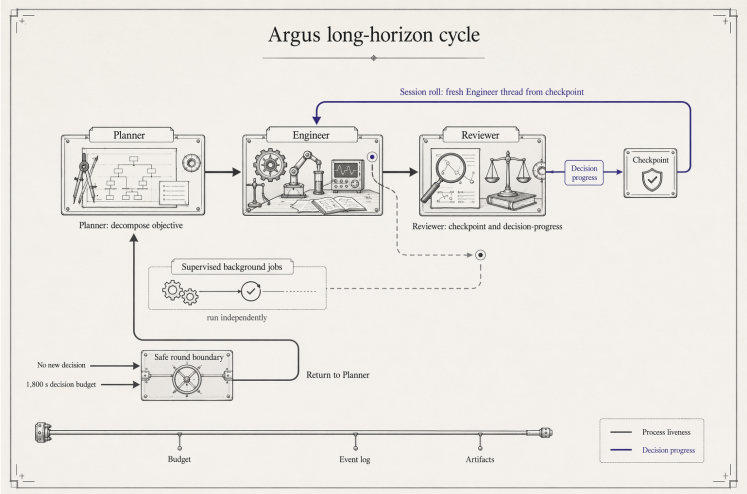


Figure 6. System schematic (not a measured performance plot) of the long-running reliability cycle: Planner, Engineer, and Reviewer run one loop; the Reviewer maintains a bounded curated checkpoint and a decision-progress class that guides the next Engineer round; supervised background jobs run alongside; and a safe round boundary returns a stalled or budget-exhausted mission to the Planner.

8.1 Decision progress, not activity

To separate real progress from busy churn, the Reviewer labels each round with a `progress_class`: `decision` or `evidence` for genuine forward motion, or `setup_only`, `artifact_sync_only`, or `none` for nondecision work. The runtime *counts* these structured labels; it never infers progress from tool activity, filenames, or token emission. Two consecutive nondecision rounds trip the stall guard, and a 1,800-second decision-progress budget — measured from the last decision or evidence increment and checked only between rounds — ends a mission that is spinning without ever cutting useful in-flight work. A separate soft/hard round escalation (12/24) catches the rarer case of a mission that makes evidence progress every round but never passes its gate, asking the Reviewer to return `blocked` and, failing that, force-ending so the Planner re-plans. Table 11 is the full guard matrix.

Guard	Trigger / action	Default	Interrupts live call?
Backend-failure recovery	failure streak → backoff, then end as error	2 / 15 s	no (round boundary)
No-progress bail	consecutive rounds with no engineer message	2	no
Decision stall	consecutive nondecision `progress_class` rounds → end	2	no
Decision-progress budget	seconds since last decision/evidence → end	1,800 s	**no**
Soft round escalation	round index → ask Reviewer to return `blocked`	12	no
Hard round escalation	round index → force-end as `blocked`	24	no
Effective-progress watchdog	no file change / non-token event → kill subprocess	3,600 s	**yes**
Runner hard-idle	no runner activity → interrupt round	3,600 s	**yes**
In-round compaction limit	auto-compactions within one round → end round	3	**yes**

Table 11. Reliability guards (`engineer/runner.py`). Round-boundary guards never sever a live call; the three in-round guards fire only against a subprocess emitting token noise or looping without effective progress. All are env-overridable and disable at 0.

8.2 In-round watchdogs and backend failure

Three guards operate *inside* a round because they target a subprocess that has stopped doing useful work. The effective-progress watchdog kills a provider subprocess that keeps emitting heartbeat or token noise for 3,600 seconds with no file change and no non-token session event; the runner hard-idle watchdog interrupts a round with no runner activity at all; and the in-round compaction guard ends a round whose session auto-compacts three times. Each default is intentionally long — research turns legitimately spend many minutes reading, planning, or waiting on model-side recovery — and each disables at 0. A backend failure streak triggers a bounded backoff and, if it persists, ends the mission as an error. A degraded reviewer backend is routed to the structured `backend_unavailable` status — treated as infrastructure and never as a `continue` — after a historical blind-gate incident in which a stale import-time schema path ran the completion gate blind for roughly ninety minutes; the reviewer now preflights its schema file and raises if it is unreadable rather than proceeding blind.

8.3 One gateway, pricing, and budget reservations

All backend execution flows through a single application-level gateway, `RunExecGateway.execute` (`core/run_gateway.py`). Token usage is priced by a small module (`core/pricing.py`, default model `gpt-5.5`) and accumulated in a *presence-aware* usage ledger (`core/usage.py`) that distinguishes a genuine zero from omitted usage, so no cost is ever fabricated from missing data. Spend control is reservation-based and event-backed: before a call runs the runtime creates a budget reservation, recorded as one of `budget.reservation.created`, `.denied`, `.settled`, or `.released`, with `budget.unpriced.blocked` and `budget.fence_breach.blocked` for the unpriceable or fence-breaching cases. This reservation-then-reconciliation cycle — settle on completion, release on abort — keeps the ledger honest, and the provider-side spend fences on a call (`max_budget_usd`, `max_ai_credits`) are injected from the reservation, not from role configuration, so a role cannot raise its own ceiling. The principal live defaults, resolved once for the CLI, daemon, and Web launch paths by the canonical `resolve_budget_caps` knob resolver (`core/knobs.py`), are a per-mission preflight cap of $30, a daily cap of $180, a global daily cap of $30 (enforced for any value above zero, so $0 disables it; some secondary `-once`/config-dataclass paths default it off instead), and at most two active daemons across projects. Beyond dollars, a provider quota permit (`core/provider_quota.py`) turns a quota denial into a paused status rather than an error.

8.4 Background jobs and the cadence wait

Not all work is short. When the Engineer starts a long experiment — for example a GPU training run — it launches a supervised background job that has its own monitor: the monitor checks health on an interval, early-stops on failure, and reports a terminal result to the operator inbox. The Engineer reads a registry of in-flight jobs (`engineer/background_subagents.py`) and classifies each as self-watched or needs-attention; when it explicitly replies `WAIT_FOR_SUBAGENT` for a healthy self-watched job, the runner yields to that job's own monitor cadence instead of busy-polling the Reviewer. It is an agent-led choice to wait, not a harness decision, and a stale or misfired sentinel simply falls back to a normal reviewed round rather than hanging the loop.

8.5 Coordinating scarce hardware: the GPU lease

On managed GPU hosts a scheduler reclaims cards that sit idle, so an operator runs a low-duty "keep-alive" loader to hold them. Argus ships an explicit *lease* tool for this (`argus_skill/tools/gpu_lease.py`): the recommended path, `gpu_lease run -- <cmd>`, frees the cards, records a lease for the job's lifetime, runs the command, and on exit re-parks the keep-alive only if no other lease is active. A `-detach` mode lets a supervisor own the lease independently of the mission, and process termination only ever targets the configured match token.

Scope of the GPU lease

The GPU lease is a real, robust *operator/agent-invoked* capability, not an automatic runtime service: nothing in the mission scheduler or engineer loop calls it implicitly. It is resource-governance tooling the agent may choose to use — which is exactly what the live integration supports, no more.

8.6 Operator cockpit and observability

Argus is not only a daemon; it is a live workbench over the same event tape the system runs on. The cockpit has a terminal surface (an Ink TUI), a web surface, and an HTTP/WebSocket API (`webapi/server.py`); there is no embedded Python REPL. The front door routes free text through the Manager, so an operator can say "switch to the Copilot backend" in plain language; the four roles are shown as peers and a mission is followed round by round via `-status`, `-watch`, and `-follow`, backed by the typed event stream. Because the Engineer and Reviewer share a work-tree and every round is persisted, the operator can open exactly what was produced and read the Reviewer's reasoning rather than a summary. The decision interface to a human is deliberately narrow: at most one blocking `operator_question` per mission, queued to and drained from an operator inbox. Most inspection endpoints are open; commands, the live WebSocket, and global metrics and trash views pass through an authentication dependency and require an `Authorization` token when one is configured — routine inspection stays friction-free while every mutation and sensitive view is gated.

8.7 Deployment, containment, and cost

The intended deployment is a per-project daemon supervised by `systemd -user` (`deploy/argus-skill.service`): it starts the daemon in the foreground so the service manager owns the lifecycle, drains to the mission boundary on stop (`TimeoutStopSec` 1800, `SIGTERM`, no mid-mission `SIGKILL`), restarts on failure with a bounded restart budget, and is cwd-bound to a project fingerprint; `loginctl enable-linger` carries it across logout and reboot. A daemon replacement or upgrade drains to the same clean boundary rather than killing a live mission, and the identity-checked resume (Section 6) ensures the restart continues the right campaign. Each builder role can run under a provider sandbox (`core/sandbox.py`) confined to the project working directory; the policy *refuses* a fake sandbox that would present as confined while remaining writable, and a sandboxed builder may never write the global state root or the provider's own configuration. Spend is bounded by the reservation system above — defaults, not guarantees: a poorly bounded campaign can still be expensive, and 7×24 operation is not free (Section 13).

ACT III · ONLINE RUNTIME EVOLUTION

Every run becomes capability—or evidence about what not to repeat.

Argus can update the operating state around a fixed model: memory, skills, tools, verifiers, routing, and evaluations.

9 Runtime Evolution Around a Fixed Model

Design objective

Let each mission update the operating state *around* a fixed model — memory, skills, tools, verifiers, routing, and evaluations — rather than the model's weights. Make every update attributable: each change names its *source* and the *authoritative owner* that commits it to a named persistence surface. That owner is often a role distinct from the one that did the work, but not always — some components are owner-complete by design. Not every component moves every run, and none of it is a gradient step on the base model.

Act II showed the machine that turns one mission into an audited increment. Act III states what that increment *is* and how it accumulates. The report's master spine (Figure 1) names the step as $H(t{+}1) = U(H(t), \text{trajectory}, \text{evidence})$; here we make H concrete from the checkpoint, skills, wiki, tools, verifiers, routing, and evaluation facts already established (Sections 5, 6, 7). Each mission transforms that state under the evidence it accepted.

$$H_t = \{M_t, S_t, A_t, V_t, R_t, Q_t\}, \qquad H_{t+1} = U(H_t, \tau_t, E_t), \qquad \theta_{t+1} = \theta_t. \tag{2}$$

Here M is memory — the curated `checkpoint.json` and the recency journal derived from the event tape; S is the versioned skill library; A is the available tool and vertical-procedure surface; V is the verifier and evidence-procedure set — the per-stage checklist and the anti-cheat construction; R is routing and role configuration — backend/model selection and campaign lifetime; and Q is the evaluation and task-definition set — the backlog and its scopes. The mission contributes a process trajectory τ_t and the accepted evidence E_t, and U folds them into the next state. The final equality carries the design commitment: $\theta_{t+1} = \theta_t$. The weights of the underlying coding-and-reasoning model are *fixed* for the campaign. Runtime evolution is **not online parameter training** — no gradient step is taken on the base model, and no claim of weight-level learning is made anywhere in this report. What changes is the operating state the fixed model reads and writes.

9.1 Every update is attributable

Runtime evolution is not one mechanism but a family, already introduced as four ownership paths — automatic, model-proposed, Reviewer-authored, and operator-invoked (Section 4). Table 12 resolves that per component: the *source* of a change to H, the *authoritative owner* that commits it, and the named surface where it persists. For the two components the system *earns* during a mission — memory (M) and skills (S) — the owner is deliberately a role other than the one that produced the work: the Reviewer certifies a checkpoint and skill edits it did not author itself, the same work-versus-certification separation that governs completion. The remaining components are owner-complete by design and honestly labeled as such. Verifiers (V) — the per-stage checklist — are **Planner-owned**: the Planner both authors and applies `checklist_ops` (`planner/planner.py`), and the Reviewer is **feedback-only**, reporting a wrong checklist through its `checklist_feedback` channel (`reviewer/_core.py`) rather than editing it; the external check on a

State component	Change source	Authoritative owner	Persistence surface
M — memory	Engineer	Reviewer	`checkpoint.json`; event-tape journal
S — skills	Engineer / Scientist	Reviewer	`skills/` versioned Markdown (`skill_ops`)
A — tools / procedures	operator / vertical	operator	installed tool surface; vertical stages
V — verifiers	Planner	Planner	per-stage checklist (`checklist_ops`); Reviewer `checklist_feedback`
R — routing / roles	operator	Manager	`continuous.json`; knob store (`CONFIG`)
Q — evaluations / tasks	Planner	scheduler	`backlog.jsonl` (`TaskSpec`)

Table 12. The runtime-state components of H: the source of each change, the authoritative owner that commits it, and the one persistence surface where it lands (with the interface object that carries it). For M and S the owner is a role distinct from the one that did the work, so the Reviewer certifies output it did not produce; V is Planner-owned (the Planner authors and applies `checklist_ops`; the Reviewer's `checklist_feedback` is an external report, not a commit gate), and A is operator-owned. Sources: Sections 5 and 6.

Planner-authored checklist is that feedback plus the Reviewer's independent completion authority, not an accept/commit gate. Tools and procedures (A) are **operator-owned** — a human outside the four-role loop installs and retires them. Routing (R) is operator-sourced and Manager-committed; evaluations (Q) are Planner-authored and scheduler-committed.

Two disciplines keep the equation honest. First, U is *partial*: a typical mission touches only a subset of H. Many missions add no new skill, synthesize no wiki page, and leave tools, routing, and evaluations untouched — A in particular is the most static component, changing mainly by operator or code action. The state is updated where accepted evidence warrants it, not rewritten wholesale each run. Second, because θ is fixed, all durable capability lives in H on disk, where it is inspectable and reversible — not latent in model weights. That is what lets the next section treat the *process*, not only the final artifact, as the asset the system actually retains.

10 The Process Is the Retained Asset

Design objective

Keep more than the answer. A completed mission yields a final artifact, but the durable value is the recorded process that produced it — the states, actions, evidence, and review feedback, together with what *changed* in the runtime state and which branches failed. Persist that process as typed, credential-redacted records, never as raw private model reasoning.

The evidence plane already records every mission as an append-only timeline (Section 7). Read as data, that timeline is strictly richer than the deliverable. Let D_{final} be the final artifact of a mission and D_{process} the recorded run that produced it.

$$D_{\text{process}} = \{(s_k, a_k, e_k, r_k, \Delta H_k)\}_{k=1}^{N} \supsetneq D_{\text{final}} = \{y^\star\}. \tag{3}$$

The strict containment is the point. The process record keeps the full per-round tuple — states, actions, evidence, feedback, and runtime-state deltas — not only the artifact $y^\star$. In each round k: s_k is the round *state* (goal, open blocker, and next step in the curated `checkpoint.json`); a_k is the *action* taken (tool and provider calls, each correlated to the tape by `call_id`); e_k is the *evidence* produced (artifacts, measurements, and provenance digests); r_k is the Reviewer's *feedback* (the verdict, its `progress_class`, and the next action); and ΔH_k is the *runtime-state delta* from Section 9 — a skill version bump, a wiki source, or a checkpoint rewrite. Crucially, D_{process} retains the branches that did *not* reach $y^\star$: dead ends recorded in the checkpoint's tried-and-failed list, and counterexamples and NO-GO findings written to the journal and wiki.

10.1 Where each part is stored

Each element lands on a mechanism already built. The typed `events.jsonl` tape holds the ordered states, actions, evidence, and verdicts; the curated `checkpoint.json` holds the current-state summary and the bounded tried-and-failed list; versioned `skills/` carry the reusable procedures learned along the way, with their history as an audit of what changed; and the per-project `wiki/` records an immutable source card after each verdict, from which the Reviewer may synthesize an evidence-cited page (Sections 6, 7). Together these make D_{process} replayable rather than anecdotal.

One boundary is deliberate. What persists is the typed, credential-redacted event record — artifacts, measurements, tool calls, and verdicts — *not* the model's raw private reasoning transcript, and this report makes no claim that such traces are published. The secret guard scrubs credentials before anything reaches the tape or the Reviewer, and provenance carries no local absolute paths, usernames, or session ids (Section 7). The retained process is an auditable public record of *what was done and shown*, held to the same redaction stance as every other persisted artifact.

11 Evidence-Gated Expansion of the Frontier

Design objective

Model capability growth as evidence-gated, not automatic. A candidate joins the retained set only when it verifies against accepted evidence; the union notation captures that intent, not a promise. In practice the set is revisable and not empirically monotone, and a run that adds no capability can still add durable value by recording what not to repeat.

The two preceding sections give a fixed-model state H that missions update (Section 9) and a retained process that outlives each artifact (Section 10). Their intended effect on the outermost spine stage — the expanded out-of-distribution frontier — is an evidence-gated capability set. Let C_t be the capabilities the system can bring to a new, out-of-distribution task at time t, and ϵ the acceptance threshold of the evidence gate.

$$C_{t+1} = C_t \cup \{c : \text{Verify}(c, E_t) \geq \epsilon\}. \tag{4}$$

Read precisely, the equation models one thing: the *retained verified additions*. A candidate capability c — a new skill, a synthesized wiki technique, a validated procedure — enters the set only when $\text{Verify}(c, E_t)$ clears the gate that the Reviewer owns (Section 5). The union is deliberately conservative about *intent*: the frontier is meant to grow by verified increments rather than by unchecked accumulation.

11.1 What the notation does not claim

The set notation is an idealization, and the report is explicit about its limits, in line with the master spine's own caption that capability is not guaranteed to grow every run. It **does not guarantee** that any given mission succeeds, that practical capability is empirically **monotone**, or that an admitted capability is retained forever. Real C_t is revisable: skills and wiki pages can be revised, archived, or retired through the ordinary lifecycle (reinforce, distill, revise, retire), so an entry can leave the effective set, and a capability verified once can regress on a harder instance. The clean union is a statement of design intent, not an empirical monotonicity claim.

Expansion also has a mode that adds no new c at all. A mission that reaches a **negative result** or a NO-GO decision still writes durable evidence about what does not work; recorded in the process (Section 10), it can reduce the repeated search cost of later missions, which is real progress even when $C_{t+1} = C_t$. Finally, two expansion directions are distinct and should not be conflated: *breadth* across domains — taking on a new arena or vertical — and *depth* within a domain — solving harder instances of a problem already in range. The public results and portfolio that follow are read against this qualified frontier, not against a promise of universal or monotone improvement (Sections 12, 12.4, 13).

ACT IV · EVIDENCE FROM THE FRONTIER

A grand thesis is only as strong as its proof points.

Six public arenas and six research programs show where Argus has produced scoped evidence—and where the evidence remains incomplete.

12 Public Evidence: Six Arenas and Six Programs

Design objective

Report every public result as a scoped claim under its exact protocol, with its evidence status attached and human or paper-reported references first where available. Units are never cross-normalized, a snapshot is never shown as a reproduced artifact, and a paper count is never shown as accepted papers.

12.1 What the evidence is for

This section is the proof-point ledger for the qualified thesis of Act III: two public records — six benchmark arenas [1] and a 41-paper research portfolio [2] — offered as evidence of *breadth* across task classes and *depth* within a few of them, not of universal capability. Nothing here is a scalar "frontier score" or a guarantee of correctness outside the stated protocols.

12.2 Six benchmark arenas

Argus publishes a live public record [1]. Figure 7 presents its six current arena results as small multiples — one panel per arena, each with its own unit — because the arenas measure different quantities (kernel rank, bits-per-byte, wall-clock seconds, solve rate, a gap score) and any shared axis would mislead. Table 13 gives the exact protocol, result, published comparison, and evidence status for each row.

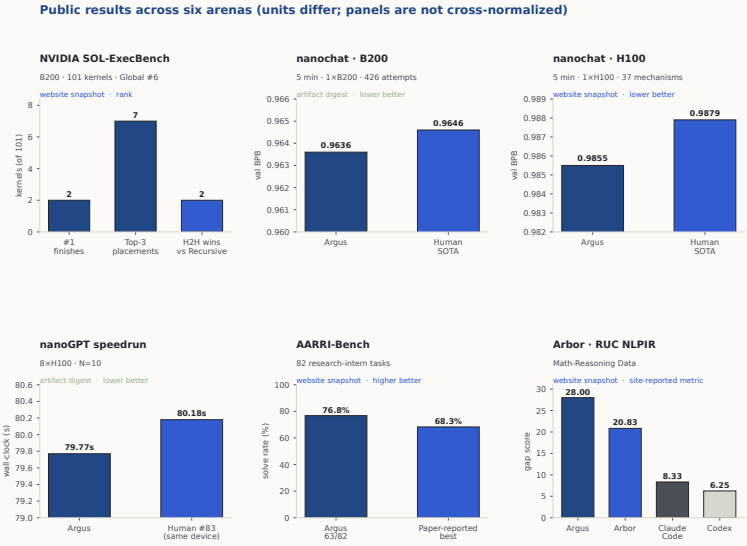


Figure 7. Public results across six arenas, drawn deterministically from `technical_report/evidence/website_results.json`. Panels are not cross-normalized; each carries its own unit and evidence status. Direction is shown where defined; Arbor retains the site's reported metric without inferring its direction. Argus bars are indigo; human/system baselines are lighter.

Arena	Protocol	Argus result	Published comparison	Status
NVIDIA SOL-ExecBench	B200; 101 kernels	Global #6; 2×#1; 7 top-3	Global rank vs. all entrants (headline)	snapshot
nanochat (B200)	5 min; 1×B200; 426 attempts	0.9636 BPB	Human SOTA 0.9646	**digest**
nanochat (H100)	5 min; 1×H100; 37 mechanisms	0.9855 BPB	Human SOTA 0.9879	snapshot
nanoGPT speedrun	8×H100; N=10	79.77 s	Same-device human #83: 80.18 s	**digest**
AARRI-Bench	82 research-intern tasks	63/82 (76.8%)	Paper-reported best 68.3%	snapshot
Arbor (RUC NLPIR)	Math-reasoning data	28.0 gap	Arbor 20.83; Claude Code 8.33; Codex 6.25	snapshot

Table 13. The six public results [1], verbatim from the evidence bundle. Human and paper-reported references are primary where available; the SOL-ExecBench headline is its global rank. SOL-ExecBench and Arbor also preserve the site's published system/agent comparisons. "digest" = committed artifact metadata; "snapshot" = public website snapshot.

Each number means exactly what its protocol row says and nothing more: the **SOL-ExecBench** [6] headline is a *global rank* against all entrants, with the two head-to-head wins over Recursive [7] the site's note, not the framing; lower bits-per-byte and fewer seconds are better on **nanochat** [5] and the **nanoGPT speedrun** [4]; and for **Arbor** the snapshot leaves the "gap" metric's direction undefined, so the values are quoted without interpreting the ordering.

12.3 Reading evidence status

Two of the six rows carry artifact-digest corroboration, stated rather than smoothed over. The **nanoGPT** 79.77 s row references an N=10 verifier-certified artifact in the external `nanogpt-speedrun-h100` project — `valid=true, n=10`, validation loss 3.2776, one-sided $p(\text{mean} < 3.28) = 0.0040$, train time 79.77 ± 0.06 s, `seal=ok`. The **nanochat B200** 0.9636 row records frozen-scorer, single-seed `MEAN_VAL_BPB`= 0.963634 with a frozen-scorer exit code of 0 and a genuine SM100 flash-attention kernel, in the `nanochat-mission-b200` project. This repository commits the logical artifact IDs, verifier excerpts, and SHA-256 digests — not the external artifact bytes. The other four rows (SOL-ExecBench, nanochat H100, AARRI-Bench, Arbor) are `website_snapshot` linked to `technical_report/evidence/website_results.json`; no corroboration file was fabricated where artifact-digest metadata was unavailable.

12.4 Six research programs

Beyond the arenas, Argus runs an optional idea-to-submission research pipeline (the `research` vertical) whose public output is a collection of **41 papers** [2] — 35 manuscripts and 6 drafts, with duplicate versions removed — across six research programs (Figure 8, Table 14).

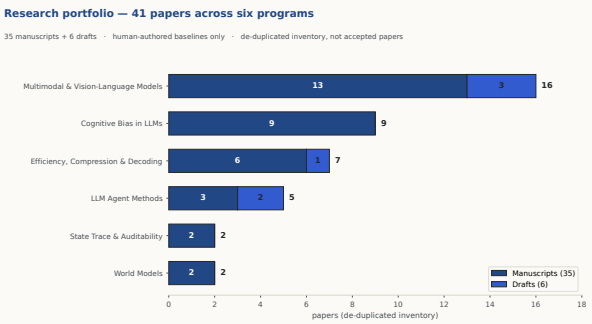


Figure 8. The 41-paper portfolio by program and status, drawn deterministically from `technical_report/evidence/paper_inventory.json`. Manuscripts are indigo, drafts lighter. The count is a de-duplicated inventory compared only to human-authored literature/baselines — not a count of accepted papers.

Program	Manuscripts	Drafts	Total
Multimodal & Vision-Language Models	13	3	16
Cognitive Bias in LLMs	9	0	9
Efficiency, Compression & Decoding	6	1	7
LLM Agent Methods	3	2	5
World Models	2	0	2
State Trace & Auditability	2	0	2
Total	**35**	**6**	**41**

Table 14. Research portfolio by program and status [2]. The machine-readable inventory is `technical_report/evidence/paper_inventory.json`.

The portfolio concentrates in a few areas — **Multimodal & Vision-Language Models** is the largest program — and each paper is positioned against human-authored literature and baselines, with the human comparison, where the site provides one, preserved in the inventory (Table 14).

12.5 What 41 means

Three qualifications keep the number honest. First, 41 is a *de-duplicated inventory* of research runs that produced a manuscript or draft, with duplicate versions removed; it is not a count of accepted papers, and this report makes no acceptance claim. Second, the 35 manuscripts and 6 drafts are reported as-is: a draft is an in-progress artifact. Third, the collection is compared *only* to human-authored literature and baselines — paper count and quality are never compared to any other agent or model.

12.6 Connection to the frontier thesis

Read against the qualified frontier of Section 11: the arena results are mostly *depth within a domain* — pushing one well-defined problem close to or past a human or paper-reported reference on stated hardware — while the portfolio is mostly *breadth across task classes*. The two are never combined into a single scalar, so this report does not claim a universal capability index, a monotone improvement curve, or that breadth and depth compose into general superiority. Each cell is a scoped claim under its own protocol and evidence tier; converting the four snapshot rows into artifact-digest records tied to a named commit is an explicit roadmap item (Section 13.2), and no row claims external peer-review acceptance.

13 Limitations and Roadmap

Argus gives an agent real judgment, real tool access, and real persistence, but does not guarantee good science; a report that documented reliability mechanisms without their boundaries would itself mislead. The limitations below are material, and the roadmap that follows pairs each with a checkable engineering objective, not a capability promise.

13.1 Limitations

Underlying model quality ceiling. Argus drives an external agent CLI (Codex, Claude Code, or Copilot CLI) and inherits its capability, availability, latency, and pricing; the quality ceiling of any run is the underlying model's. Argus enforces integrity of *measurement*, not wisdom of *goal*, and guarantees neither novelty, correctness, nor state of the art.

Single fallible Reviewer authority. The Reviewer is the sole completion authority, and it is a model: a wrong Reviewer can accept an incomplete result or reject a finished one. Argus mitigates this with a shared work-tree, an execution-log audit, and a structured checklist (Section 7), and routes a degraded reviewer backend to an explicit infrastructure-failure status rather than a `continue` — but it cannot eliminate model error, and the historical blind-gate incident (Section 8) is this concentration's concrete failure mode.

Two digest-correlated results versus four snapshots. Only two of the six public results (Section 12) carry committed artifact-digest records; the other four are website snapshots linked to the evidence bundle. All are real, scoped claims, but the snapshots are not yet reproducible from a named commit, and the report does not present them as if they were.

External artifact bytes not committed here. Even the two digest rows commit logical arti-

fact IDs, verifier excerpts, and SHA-256 digests — not the external artifact bytes, which live in named external projects, so third-party reproduction of those rows depends on those projects.

Benchmark-specific integrity. The anti-cheat construction is task-agnostic, but a determined agent explores each benchmark's exploit surface, and a new arena can present a new exploit — a leaked test set, an unrandomized input, a side channel — the current checks were not written for. Measurement integrity is ongoing, not solved.

Cost and compute. Long-horizon autonomy consumes provider tokens continuously. Budgets and reservations bound spend, but 7×24 operation is not free.

GPU lease is explicit, not automatic. The GPU lease (Section 8) is robust but operator/agent-invoked, not an automatic runtime service; coordinating scarce hardware still depends on the agent choosing to use it.

Runtime state can be wrong, stale, revised, or retired. The evolving state H (Section 9) is not guaranteed correct: a skill, wiki page, verifier, or route can be mistaken, go stale, or be revised or retired through the ordinary lifecycle. It lives on disk precisely so it can be inspected and rolled back, not because it is assumed permanent.

No guarantee of monotone capability expansion. The capability set C_t (Section 11) is not empirically monotone: a mission can add no new capability, and a capability verified once can regress on a harder instance. The union notation is design intent, not a promise that every run expands the frontier.

13.2 Roadmap

Each objective below answers a limitation above and is stated so its completion is checkable, not a capability promise — the highest priority being to turn snapshots into reproducible evidence while keeping exactly one owner of completion.

Limitation	Checkable objective
Two digest records vs. four snapshots	Convert SOL-ExecBench, nanochat H100, AARRI-Bench, and Arbor into artifact-digest records tied to a named commit, so every published number is third-party-verifiable from a commit and an artifact digest.
External artifact bytes not committed	Publish or reference retrievable artifact bundles for each digest row, so reproduction does not depend on private external projects.
Benchmark-specific integrity	Extend the anti-cheat construction and the Reviewer's structured stage checklists as each new arena reveals its exploit surface, without adding a second family of validators that could disagree with the completion authority.
Single fallible Reviewer authority	Reduce Reviewer error through richer execution-log auditing and evidence-grounded verdicts, and harden detection of a degraded reviewer backend so a blind gate cannot recur.
Runtime state wrong, stale, or retired	Strengthen lifecycle review so mistaken or stale skills, pages, and verifiers are caught and retired on evidence, with the version history serving as the audit.
Cost, compute, and explicit GPU lease	Tighten budget accounting and cockpit cost attribution, and evaluate tighter but still explicit GPU-lease scheduling, to make a 7×24 campaign cheaper to run and steer.
Model ceiling and non-monotonicity	Track per-arena and per-program evidence over time so depth and breadth are reported as measured, never as a guaranteed monotone capability curve.

Table 15. Each roadmap objective answers a limitation from Section 13 and is stated so its completion is checkable.

The through-line

Argus is designed to make research intelligence denser in time, research experience more reusable, and the frontier of tractable work wider. The claim remains conditional on evidence: no run, capability, or number outruns the protocol that supports it.

A Formal Notation

The report's explanatory symbols are collected here for reference; they summarize design intent, not reported benchmark metrics — $\rho_{\mathrm{DI}}(T)$ in particular is a lens, not an instrument (Section 2).

Symbol	Meaning
$\rho_{\mathrm{DI}}(T)$	explanatory dense-intelligence density over wall-clock horizon T
$\lambda(t)$	rate of relevant reasoning and tool-mediated research work
η_d, η_x, η_v	decision, execution, and verification effectiveness factors
H_t	runtime capability state $\{M_t, S_t, A_t, V_t, R_t, Q_t\}$
τ_t, E_t	process trajectory and accepted evidence from run t
θ_t	underlying model parameters
C_t	retained verified capability set
ϵ	evidence threshold in the explanatory capability-set model

B Key Interfaces and Status Taxonomy

The role decision interfaces are summarized in Section 5 (Tables 5–8); this appendix records the status taxonomy they resolve into. Three enumerations govern outcomes: `ReviewStatus` (the verdict), `LoopStatus` (a mission's terminal or paused outcome), and `PlannerVerdictStatus` (the planner's event contract), the last carrying an explicit (success, recoverable) policy that determines how the scheduler reacts.

PlannerVerdictStatus	success	recoverable	Meaning
`planned`	yes	no	a new task batch was produced
`completed`	yes	no	the project is certified complete
`research_incomplete`	no	yes	more research is required
`paused_budget`	no	yes	budget exhausted; recheck later
`paused_no_breakthrough`	no	yes	no breakthrough this cycle
`exhausted_current_-methods`	no	yes	current methods exhausted
`provider_cooldown`	no	yes	provider cooldown in effect
`provider_fence`	no	yes	provider spend fence hit
`infra_blocked`	no	yes	infrastructure blocker
`error`	no	no	unrecoverable error

Table 16. `PlannerVerdictStatus` and its (success, recoverable) policy (`core/planner_verdict.py`). Legacy aliases (`done`, `paused_provider_cooldown`, `paused_provider_fence`) map onto these.

ReviewStatus (`core/models.py`): `done`, `continue`, `blocked`, plus the research-pause states `research_incomplete`, `paused_no_breakthrough`, and `exhausted_current_methods`.

LoopStatus (`core/models.py`): terminal statuses `done`, `max_rounds`, `blocked`, `no_progress`, `error`, `budget_exhausted`; paused statuses `paused_budget`, `paused_provider_cooldown`, `paused_provider_-fence`, `infra_blocked`, `research_incomplete`, `paused_no_breakthrough`, `exhausted_current_methods`; and the control-transfer status `replan_requested`, used only when dynamic planning requests a return to the Planner. Every mission yields one of these — never an unbounded spin (Section 6).

C Persisted State Map

Table 9 (Section 6) lists the principal state files: the root layout is in `core/paths.py` and the per-project checkpoint path in `apps/_runtime.py`. The global root `~/.argus-skill/` (overridable via `ARGUS_SKILL_HOME`; a legacy `ARGUS_SKILL_LIFE_DIR` shim is kept for migration) holds cross-project identity, the shared skill library, and an operator knob store; each `projects/<fingerprint>/` subtree holds the append-only `events.jsonl`, `backlog.jsonl`, `continuous.json`, `inbox.jsonl`, a `skills/` directory, `daemon.pid`, `daemon.status.json`, and `checkpoint.json`. A path resolver rejects unresolved shell placeholders and empty, `.`, or `/` fingerprints, so a mis-resolved path fails loudly rather than writing to the wrong place.

D Event Taxonomy

Cross-component state flows through a canonical catalog of **112 typed event types** across **11 categories** (`core/event_catalog.py`), of which **75** carry a validated payload schema (Table 17).

Category	Count	Representative events
lifecycle	40	loop.start, round.main.completed, round.review.completed, budget.reservation.*, life.mission.started
skill	20	skill.match.*, scientist.start, skill.outcome
wiki	17	wiki source/page ingest, promotion, validation
planner	9	life.planner.start, life.planner.verdict
research	7	research.achievement.certified (sole authoritative achievement)
agent_io	5	agent.io.start, agent.io.stream, agent.io.complete
daemon	5	daemon start / stop / handoff / status
idea	3	idea lifecycle
provider	3	provider.request.started/completed/denied
usage	2	usage.recorded, cost
operator	1	operator inbox event

Table 17. Event taxonomy: 112 types across 11 categories (core/event_catalog.py). Budget, loop, and round events are grouped under lifecycle.

E Evidence Map

Every engineering claim in this report is grounded in the current argus-skill repository [3] rather than in transient run directories. Table 18 maps each subsystem to its primary live source.

Subsystem (section)	Primary source path(s)
Three planes (§5)	docs/ARCHITECTURE.md; core/, manager/, engineer/, reviewer/
Mission lifecycle / daemon (§6)	life/supervisor/_core.py, daemon/life_worker.py, daemon/handoff.py
State & memory (§6)	core/paths.py, apps/_runtime.py, engineer/checkpoint.py, life/memory.py, skills/, wiki/
Role interfaces (§5)	core/models.py, planner/planner.py, manager/_core.py, reviewer/reviewer_schema.json
Execution / budgets (§8)	core/run_gateway.py, core/pricing.py, core/usage.py, core/provider_quota.py
Reliability guards (§8)	engineer/runner.py, regime_jump/
Evidence & integrity (§7)	core/event_catalog.py, core/secret_guard.py, skills/provenance.py
Cockpit / API (§8)	webapi/server.py
Deployment / security (§8)	deploy/argus-skill.service, core/sandbox.py, core/vault_preflight.py
Public results & portfolio (§12–12.4)	technical_report/evidence/website_results.json, paper_inventory.json

Table 18. Evidence map: subsystem to live source. This report cites repository paths on main, not any transient local filesystem location.

Figure provenance. The four deterministic figures (system_planes, mission_lifecycle, public_results, paper_portfolio) come from technical_report/figures/build_report_figures.py with no image-model call and SHA-256 digests in figures/REPORT_FIGURES.json; the two editorial plates carry image-2 prompt, inspect, review, and provenance sidecars in figures/IMAGE2_FIGURES.json.

Paper-inventory summary. The 41-paper portfolio (Section 12.4) — 35 manuscripts and 6 drafts across the six programs of Table 14 — is enumerated in technical_report/evidence/paper_inventory.json, compared only to human-authored literature.

References

[1] Argus Team. Argus public results. Website snapshot, https://argusbot.cn/results.html, 2026.

[2] Argus Team. Argus research-paper collection. Website snapshot, https://argusbot.cn/research.html, 2026.

[3] Argus Team. argus-skill: A 7×24 system for long-horizon autonomous research. GitHub repository, 2026.

[4] Andrej Karpathy. nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs. GitHub repository, 2022.

[5] Andrej Karpathy. nanochat: The best ChatGPT that $100 can buy. GitHub repository, 2025.

[6] Anne Ouyang et al. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025.

[7] Recursive. First steps toward automated AI research. GitHub repository, 2026.